Computation Theory — Exam-Checklist Worksheet

Problems, hints & solutions for each point of the `comp_theory_8020` checklist

Part IB — Cambridge CST — Paper 6

Cover each Solution box first. These questions reward precise definitions and correct constructions.

---

1 Equivalence & the Church–Turing thesis

Warm-up 1. Name the formalisations of "algorithm" that the course proves equivalent.

Hint. Three machine/function models plus one from Church.

Solution

Register machines, Turing machines, partial recursive functions, and λ-definable functions — all compute the same class ("computable").

Warm-up 2. Is the Church–Turing thesis a theorem? Justify.

Hint. What kind of statement is "every algorithm is…"?

Solution

No. "Intuitively computable" is informal, so the thesis can't be proved; it's a well-supported *claim*, evidenced by the convergence of independently-defined models on the same function class.

Exam-style. Explain what the equivalence of these models means, and why it underpins the proof that some problems are undecidable.

Hint. Two payoffs: robustness of "computable", and "programs as data".

Solution

Meaning: each model computes *exactly the same* set of partial functions $\mathbb{N} \rightharpoonup \mathbb{N}$; a construction in one transfers to the others. So "computable" is *model-independent* — we can prove results in whichever model is most convenient (register machines for coding, λ-calculus for higher-order constructions).

Why undecidability follows: because programs can be **coded as numbers** (data), a single *universal* machine can run any program, and a **diagonal** argument then exhibits a problem (the Halting Problem) that *no* machine in the class decides — and by the equivalence, no algorithm at all.

2 λ-calculus

Warm-up 1. β-reduce $(\lambda x.x\,x)(\lambda y.y)$ to normal form.

Hint. Substitute the argument for x, then reduce.

Solution

$(\lambda x.x\,x)(\lambda y.y) \to_\beta (\lambda y.y)(\lambda y.y) \to_\beta \lambda y.y = \mathrm{I}$.

Warm-up 2. Compute $(\lambda y.x)[y/x]$ correctly.

Hint. Substituting y for x would be captured by the binder λy — rename first.

Solution

α-rename the bound y first: $(\lambda z.x)[y/x] = \lambda z.y$ *(not* $\lambda y.y$*)*. Substitution is capture-avoiding.

Exam-style. Reduce $\mathbf{Succ}\,\underline{1}$ to normal form and confirm it is $\underline{2}$; then explain why $\Omega = (\lambda x.x\,x)(\lambda x.x\,x)$ has no normal form, and what the Church–Rosser theorem guarantees.

Hint. $\mathbf{Succ} = \lambda nfx.f\,(n\,f\,x)$ *and* $\underline{1} = \lambda fx.f\,x$, $\underline{2} = \lambda fx.f\,(f\,x)$.

Solution

$$\mathbf{Succ}\,\underline{1} = (\lambda nfx.f(nfx))(\lambda fx.fx) \to_\beta \lambda fx.f\big((\lambda fx.fx)\,f\,x\big) \twoheadrightarrow_\beta \lambda fx.f(fx) = \underline{2}.$$

$\Omega \to_\beta (\lambda x.xx)(\lambda x.xx) = \Omega$ — every reduction reproduces Ω, so it *never* reaches a redex-free term; no normal form. **Church–Rosser (confluence):** if $M \twoheadrightarrow_\beta M_1$ and $M \twoheadrightarrow_\beta M_2$ then both reduce to a common term; consequently a normal form, *if it exists*, is unique up to α-equivalence (and normal-order reduction will find it).

3 λ-definability

Warm-up 1. State the two conditions for a closed term F to represent a partial function f.

Hint. *One for "defined", one for "undefined".*

Solution

If $f(\vec{x})\!=\!y$ then $F\,\underline{x_1}\cdots\underline{x_n} =_\beta \underline{y}$; if $f(\vec{x})\!\uparrow$ then $F\,\underline{x_1}\cdots\underline{x_n}$ has *no* β*-normal form.*

Warm-up 2. State the theorem relating computability and λ-definability.

Hint. *Iff.*

Solution

A partial function is computable *iff* it is λ-definable.

Exam-style. Show that addition is λ-definable: take $P \equiv \lambda m\,n\,f\,x.\,m\,f\,(n\,f\,x)$ and verify $P\,\underline{m}\,\underline{n} =_\beta \underline{m{+}n}$.

Hint. *Use* $\underline{k}\,f\,x =_\beta f^k x$, *and* $f^m(f^n x) = f^{m+n}x$.

Solution

$$\begin{aligned} P\,\underline{m}\,\underline{n} &=_\beta \lambda f\,x.\ \underline{m}\,f\,(\underline{n}\,f\,x) \\ &=_\beta \lambda f\,x.\ \underline{m}\,f\,(f^n x) \\ &=_\beta \lambda f\,x.\ f^m(f^n x) \\ &=_\beta \lambda f\,x.\ f^{\,m+n}x \ =\ \underline{m{+}n}. \end{aligned}$$

So P represents $+$; since $+$ is total, the "undefined" clause is vacuous. (For partial functions you must additionally ensure non-termination on undefined inputs.)

4 Register machines

Warm-up 1. Write the three instruction forms of a register machine.

Hint. *Increment-and-jump, decrement-or-branch, halt.*

Solution

$R^+ \to L'$ (add 1 to R, go to L'); $R^- \to L', L''$ (if $R{>}0$ subtract 1 and go to L', else go to L''); HALT.

Warm-up 2. Why does copying register S into D require a temporary register?

Hint. What does reading S do to it?

Solution

You can only inspect S by decrementing it (which destroys it). So you decrement S into *both* D and a temp T, then move T back to restore S.

Exam-style. Write a register machine that sets $R_0 := R_1 + R_2$ (you may destroy R_1, R_2; assume $R_0 = 0$ initially). Give the labelled program.

Hint. Repeatedly decrement R_1 while incrementing R_0; then do the same for R_2.

Solution

$$\begin{aligned} L_0 &: \ R_1^- \to L_1,\ L_2 && \text{(drain } R_1 \text{ into } R_0\text{)} \\ L_1 &: \ R_0^+ \to L_0 \\ L_2 &: \ R_2^- \to L_3,\ L_4 && \text{(drain } R_2 \text{ into } R_0\text{)} \\ L_3 &: \ R_0^+ \to L_2 \\ L_4 &: \ \mathsf{HALT} \end{aligned}$$

On halting, $R_0 = R_1^{\text{init}} + R_2^{\text{init}}$ and $R_1 = R_2 = 0$.

5 Undecidability

Warm-up 1. Define "decidable set" and "recursively enumerable (r.e.) set".

Hint. One always halts; the other halts on members.

Solution

Decidable: its characteristic function is computable (an algorithm answers "$x \in S$?" and always halts). **r.e.:** it is the domain of a computable partial function (a machine halts on exactly the members of S).

Warm-up 2. Complete: a set S is decidable iff _____.

Hint. Think about S and its complement.

Solution

…iff both S and its complement $\overline{S}$ are r.e.

Exam-style. Prove the Halting Problem is undecidable by diagonalisation; then sketch how undecidability of another problem follows by reduction.

Hint. Assume a total decider H, build a machine that contradicts it on its own code.

Solution

Suppose a total computable H exists with $H(A, D) = 1$ iff program A halts on input D. Define

$$C(A): \ \text{compute } H(A, A);\ \text{if } H(A, A) = 0 \text{ return } 1,\ \text{else loop forever.}$$

Then $C(A)\downarrow \iff H(A, A) = 0 \iff A(A)\uparrow$. Feed C its own code: $C(C)\downarrow \iff C(C)\uparrow$ — contradiction. So no such H exists.

By reduction: to show "does P halt on empty input?" is undecidable, map any Halting instance (A, D) to a program $P_{A,D}$ that ignores its input and runs A on D. Then $P_{A,D}$ halts on empty input iff A halts on D. A decider for the empty-input problem would thus decide Halting — impossible.

6 Recursive functions

Warm-up 1. Name the three basic functions of the primitive recursive functions.

Hint. Constant, increment, select.

Solution

Zero ($\mathbf{0}$), successor (succ), and the projections (proj_i^n).

Warm-up 2. Which closure operation turns primitive recursive into *partial* recursive?

Hint. Unbounded search.

Solution

Minimisation, $\mu y\,[f(\vec{x}, y) = 0]$ (search for the least such y; may not halt $\Rightarrow$ partial).

Exam-style. Define addition by primitive recursion from succ; then explain why Ackermann's function is computable but *not* primitive recursive.

Hint. Primitive recursion: base case on 0, step uses the previous value. For Ackermann, think about growth rate vs the p.r. functions.

Solution

$$\mathrm{add}(x, 0) = x, \qquad \mathrm{add}(x, y{+}1) = \mathrm{succ}\big(\mathrm{add}(x, y)\big).$$

Ackermann: it is *total* and clearly *computable* (a terminating, well-founded recursion). But it grows faster than *every* primitive recursive function — one can show that for any p.r. f there is an argument beyond which Ackermann dominates f (a diagonal argument over an enumeration of the p.r. functions). Hence it cannot itself be p.r. **Moral:** primitive recursion (bounded loops) does not capture all total computable functions; minimisation / unbounded recursion is strictly more powerful.

Re-do the Halting proof, a β-reduction, and the register-machine program cold before the exam.